ISIM
Institut Supérieur d'Informatique de Médenine

Année Universitaire
2025-2026

Travaux Pratiques - Développement Web Avancé

TP N°4 : Modélisation Base de Données TaskFlow

Classe : MP1ILC **Enseignant :** M. JARRAY Mondher **Durée :** 1h30 **Projet :** TaskFlow

Objectifs du TP

- Configurer la connexion à la base de données
- Créer les migrations pour les tables de TaskFlow
- Définir les relations entre les tables
- Créer des seeders pour les données de test
- Créer des factories pour générer des données

Prérequis

- TP 1-3 terminés (projet TaskFlow avec routes et vues)
- MySQL/MariaDB installé (Laragon, XAMPP, ou WAMP)
- Comprendre les concepts de migrations (Chapitre 4)

Partie 1 : Configuration de la Base de Données (15 min)

Exercice 1.1 : Créer la base de données — 2 pts

Option A : Via phpMyAdmin

1. Ouvrez phpMyAdmin (`http://localhost/phpmyadmin`)
2. Cliquez sur "Nouvelle base de données"
3. Nom : `taskflow`
4. Interclassement : `utf8mb4_unicode_ci`

Option B : Via le terminal MySQL

```sql
CREATE DATABASE taskflow CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
```

Exercice 1.2 : Configurer le fichier .env

2 pts

Ouvrez le fichier `.env` et modifiez les lignes suivantes :

```env
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=taskflow
DB_USERNAME=root
DB_PASSWORD=
```

> **Note**
>
> Si vous utilisez WAMP, le mot de passe par défaut est vide. Si vous utilisez MAMP, le mot de passe est souvent `root`.

Testez la connexion :

```
$ php artisan migrate:status
```

Si la connexion fonctionne, vous verrez un tableau vide (pas d'erreur).

Partie 2 : Création des Migrations (35 min)

Exercice 2.1 : Migration de la table Projets

5 pts

Créez la migration :

```
$ php artisan make:migration create_projets_table
```

Complétez le fichier créé :

```php
public function up(): void
{
    Schema::create('projets', function (Blueprint $table) {
        $table->id();
        $table->string('nom');
        $table->text('description')->nullable();
        $table->enum('statut', ['en_attente', 'en_cours', 'termine'])
              ->default('en_attente');
        $table->date('date_debut')->nullable();
        $table->date('date_fin')->nullable();
        $table->foreignId('user_id')
              ->constrained()
              ->onDelete('cascade');
        $table->timestamps();
    });
}

public function down(): void
{
    Schema::dropIfExists('projets');
}
```

Questions :

1. Pourquoi utilise-t-on `->nullable()` pour la description ?

2. Que fait `->onDelete('cascade')` ?

Exercice 2.2 : Migration de la table Tâches

5 pts

Créez la migration :

```
$ php artisan make:migration create_taches_table
```

Complétez la migration avec les colonnes suivantes :

- **a** `id` : clé primaire
- **b** `titre` : string, obligatoire
- **c** `description` : text, nullable
- **d** `statut` : enum (todo, in_progress, done), défaut 'todo'
- **e** `priorite` : enum (basse, moyenne, haute), défaut 'moyenne'
- **f** `date_echeance` : date, nullable
- **g** `projet_id` : clé étrangère vers projets (cascade)
- **h** `user_id` : clé étrangère vers users (nullable, set null)
- **i** `timestamps` : created_at et updated_at

Écrivez votre code de migration ici :

Exercice 2.3 : Exécuter les migrations

3 pts

Exécutez toutes les migrations :

```
$ php artisan migrate
```

Vérifiez le résultat :

```
$ php artisan migrate:status
```

Capture d'écran : Résultat de migrate:status montrant les tables créées

Vérifiez dans phpMyAdmin que les tables suivantes existent :

- users
- projets
- taches
- migrations (table système)

Partie 3 : Seeders et Factories (25 min)

Exercice 3.1 : Créer un UserSeeder

3 pts

```
$ php artisan make:seeder UserSeeder
```

Complétez database/seeders/UserSeeder.php :

```php
use App\Models\User;
use Illuminate\Support\Facades\Hash;

public function run(): void
{
    // Créer un utilisateur admin
    User::create([
        'name' => 'Admin TaskFlow',
        'email' => 'admin@taskflow.com',
        'password' => Hash::make('password123'),
    ]);

    // Créer quelques utilisateurs de test
    User::create([
        'name' => 'Ahmed Ben Ali',
        'email' => 'ahmed@example.com',
        'password' => Hash::make('password123'),
    ]);

    User::create([
        'name' => 'Fatma Trabelsi',
        'email' => 'fatma@example.com',
        'password' => Hash::make('password123'),
    ]);
}
```

Exercice 3.2 : Créer un ProjetSeeder

4 pts

```
$ php artisan make:seeder ProjetSeeder
```

Créez au moins 5 projets de test :

```php
use App\Models\Projet;

public function run(): void
{
    Projet::create([
        'nom' => 'Site E-commerce ISIM',
        'description' => 'Développement d\'une boutique en ligne pour les produits ISIM',
        'statut' => 'en_cours',
        'date_debut' => '2025-01-15',
        'user_id' => 1,
    ]);

    // Ajoutez 4 autres projets...
}
```

Complétez avec 4 autres projets :

Exercice 3.3 : Créer des Factories

5 pts

Créez la factory pour les projets :

```
$ php artisan make:factory ProjetFactory --model=Projet
```

```php
// database/factories/ProjetFactory.php
public function definition(): array
{
    return [
        'nom' => fake()->sentence(3),
        'description' => fake()->paragraph(),
        'statut' => fake()->randomElement(['en_attente', 'en_cours', 'termine']),
        'date_debut' => fake()->dateTimeBetween('-2 months', 'now'),
        'date_fin' => fake()->optional()->dateTimeBetween('now', '+3 months'),
        'user_id' => \App\Models\User::factory(),
    ];
}
```

Créez la factory pour les tâches :

```
$ php artisan make:factory TacheFactory --model=Tache
```

Écrivez le code de TacheFactory :

Exercice 3.4 : DatabaseSeeder principal

3 pts

Modifiez database/seeders/DatabaseSeeder.php :

```php
public function run(): void
{
    // 1. Créer les utilisateurs
    $this->call(UserSeeder::class);

    // 2. Créer les projets manuels
    $this->call(ProjetSeeder::class);

    // 3. Créer des projets avec factory (10 projets aléatoires)
    \App\Models\Projet::factory(10)->create([
        'user_id' => 1, // Attribuer à l'admin
    ]);

    // 4. Créer des tâches pour chaque projet
    \App\Models\Projet::all()->each(function ($projet) {
        \App\Models\Tache::factory(rand(3, 8))->create([
            'projet_id' => $projet->id,
        ]);
    });
}
```

Exécutez le tout :

```
$ php artisan migrate:fresh --seed
```

Capture d'écran : phpMyAdmin montrant les données insérées

Barème d'évaluation

Création BDD (1.1)	2 pts
Configuration .env (1.2)	2 pts
Migration projets (2.1)	5 pts
Migration tâches (2.2)	5 pts
Exécution migrations (2.3)	3 pts
UserSeeder (3.1)	3 pts
ProjetSeeder (3.2)	4 pts
Factories (3.3)	5 pts
DatabaseSeeder (3.4)	3 pts
Total	**32 pts**

☆ Exercice Bonus (+4 pts)

1. Créez une migration pour ajouter une colonne `couleur` à la table projets
2. Créez une table `commentaires` liée aux tâches
3. Ajoutez une factory pour les commentaires

Livrables à rendre

- Fichiers de migrations (create_projets_table, create_taches_table)
- Fichiers seeders (UserSeeder, ProjetSeeder, DatabaseSeeder)
- Fichiers factories (ProjetFactory, TacheFactory)
- Captures d'écran de phpMyAdmin
- Réponses aux questions